

structs.go

```
1 package main
2
3 import (
4     "fmt"
5     "time"
6 )
7
8 // create a struct type called user that has three fields: firstName, last
// Name, and birthdate. All of these fields should be of type string. A struc
// t can be created either outside of a function or inside of a function. In
// this case, we will create the struct type outside of the main function. Th
// is is standard practice in Go, as it allows the struct type to be used thr
// oughout the entire package. The struct type is defined using the type keyw
// ord, followed by the name of the struct (user), and then the struct fields
// are defined within curly braces. Each field has a name and a type, separa
// ted by a space. In this case, all three fields are of type string.
9 type user struct {
10     firstName string
11     lastName  string
12     birthdate string
13     createdAt time.Time // Time is also a struct type, and it is part of t
// he time package. The time package is part of the Go standard library, and
// it provides functionality for working with dates and times. The createdAt
// field will be used to store the date and time when the user was created. T
// his field is of type time.Time, which is a struct type that represents a s
// pecific point in time. The time.Time struct has many methods that can be u
// sed to manipulate and format dates and times.
14 }
15
16 func main() {
17     firstName := getUserData("Please enter your first name: ")
18     lastName  := getUserData("Please enter your last name: ")
19     birthdate := getUserData("Please enter your birthdate (MM/DD/YYYY): ")
20
21     var appUser user // create a variable of type user, which is a struct
// type. The variable is named appUser, and it will be used to store the data
// that we gather from the user. The variable is created using the var keywo
// rd, followed by the name of the variable (appUser), and then the type of t
// he variable (user). The variable is initialized with the zero value for th
// e user struct type, which means that all of its fields will be set to thei
// r zero values. In this case, all of the fields will be set to empty string
// s, except for the createdAt field, which will be set to the zero value for
// time.Time, which is January 1, year 1, 00:00:00 UTC.
22 }
```

```

23     appUser = user{
24         firstName: firstName,
25         lastName:  lastName,
26         birthdate: birthdate,
27         createdAt: time.Now(),
28     }
29
30     var appUser1 user = user{} // Declare and Initialize the struct
31
32     appUser1.firstName = firstName
33     appUser1.lastName = lastName
34
35     outputUserDetailsPointer(&appUser1)
36
37     // ... do something awesome with that gathered data!
38
39     outputUserDetails(appUser)
40
41     fmt.Println(firstName, lastName, birthdate)
42 }
43
44 func outputUserDetailsPointer(u *user) {
45     fmt.Println("Pointer Details: ", u.firstName, (*u).lastName) // Rememb
er with pointers you must dereference them to access their values but Go a
llows you to use a shortcut method to access the underlying field without
explicitly dereferencing the pointer by u.firstName
46 }
47
48 // This function prints the fields within the struct user. Fields are diff
erent from properties which use Getters and Setters to access the underlyi
ng values.
49 func outputUserDetails(u user) {
50     fmt.Println(u.firstName, u.lastName, u.createdAt)
51 }
52
53 func getUserData(promptText string) string {
54     fmt.Print(promptText)
55     var value string
56     fmt.Scan(&value)
57     return value
58 }
59

```